CS 4400: Artificial Intelligence

Propositional Logic

Contents

1. Knowledge-based agents
2. The Wumpus world
3. Logical systems
4. Propositional logic
5. Theorem proving
6. Resolution
7. PL agent

4/7. Propositional logic

- Propositional logic is one of the simplest and intuitive logical systems (Ch 8)
- There are more powerful logical systems such as first-order logic (Ch 9)

- Propositional logic
  - Syntax: how to write a sentence as a symbol or a combination of symbols
  - Semantics: whether a sentence is true or false

Propositional logic

- Propositional logic is the simplest logic – illustrates basic ideas

- The proposition symbols P_1, P_2 etc are sentences

  - If S is a sentence, $\neg S$ is a sentence (negation)
  - If S_1 and S_2 are sentences, $S_1 \wedge S_2$ is a sentence (conjunction)
  - If S_1 and S_2 are sentences, $S_1 \vee S_2$ is a sentence (disjunction)
  - If S_1 and S_2 are sentences, $S_1 \Rightarrow S_2$ is a sentence (implication)
  - If S_1 and S_2 are sentences, $S_1 \Leftrightarrow S_2$ is a sentence (biconditional)

Syntax of propositional logic

- *Sentence* = *AtomicSentence* | *ComplexSentence*
- *AtomicSentence* = *True* | *False* | P | Q | R |…
- *ComplexSentence* = (*sentence*) | [*Sentence*]

  | ~*Sentence* : negation

  | (*Sentence* $\wedge$ *Sentence*) : conjunction

  | (*Sentence* $\vee$ *Sentence)* : disjunction

  | (*Sentence* $\Rightarrow$ *Sentence*) : implication

  | (*Sentence* $\Leftrightarrow$ *Sentence*) : biconditional
- In $\alpha \Rightarrow \beta$, α is called premise or antecedent, β is called conclusion or consequent
- ~, $\wedge$, $\vee$, $\Rightarrow$, $\Leftrightarrow$ are logical connectives of PL

Semantics

- The syntax of a sentence says nothing about its truth value
  - P: “I have 10 dollars in my pocket”
  - Is P true? Maybe yes, maybe no
- Semantics specify how to compute the truth value of any sentence given a model

Semantics

Each model/world specifies true or false for each proposition symbol

E.g. $P_{1,2}$ $P_{2,2}$ $P_{3,1}$
false true false

With these symbols, 8 possible models, can be enumerated automatically.

Rules for evaluating truth with respect to a model *m*:

$\neg S$	is true iff	S is false	
$S_1 \wedge S_2$	is true iff	S_1 is true and	S_2 is true
$S_1 \vee S_2$	is true iff	$S_1$is true or	S_2 is true
$S_1 \Rightarrow S_2$	is true iff	S_1 is false or	S_2 is true
i.e.,	is false iff	S_1 is true and	S_2 is false
$S_1 \Leftrightarrow S_2$	is true iff	$S_1 \Rightarrow S_2$ is true and	$S_2 \Rightarrow S_1$ is true

Simple recursive process evaluates an arbitrary sentence, e.g.,

$$\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1}) = \textit{true} \wedge (\textit{true} \vee \textit{false}) = \textit{true} \wedge \textit{true} = \textit{true}$$

Semantics

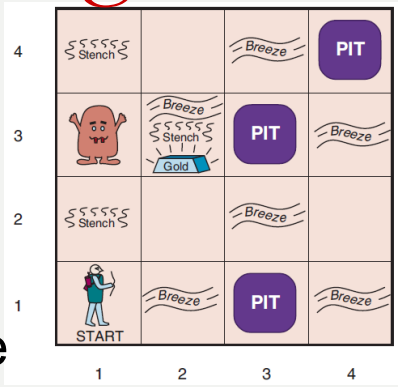
- Atomic sentences: assigned directly by model
  - "P" is true in models where P ≡ *True*
- Complex sentences: truth assigned by rules
  -

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	*false*	*true*	*false*	*false*	*true*	*true*
false	*true*	*true*	*false*	*true*	*true*	*false*
true	*false*	*false*	*false*	*true*	*false*	*false*
true	*true*	*false*	*true*	*true*	*true*	*true*

  - Rows are models, columns are sentences
  - Preview of logical equivalents
    - $P \Rightarrow Q \equiv \sim P \vee Q$
    - $P \Leftrightarrow Q \equiv (P \Rightarrow Q) \wedge (Q \Rightarrow P)$

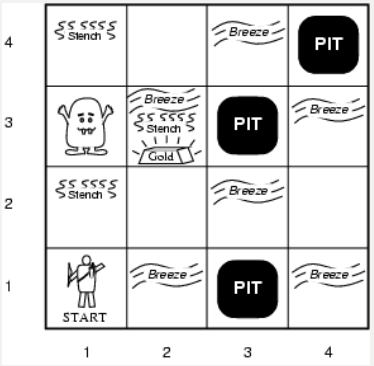
Implication is always true when the premises are False!

Wumpus world in propositional logic



- Symbols
  - $P_{[x,y]}$ is true iff there is a pit in [x,y]
  - $W_{[x,y]}$ is true iff there is a wumpus in [x,y], dead or alive
  - $B_{[x,y]}$ is true iff the agent perceives a breeze in [x,y]
  - $S_{[x,y]}$ is true iff the agent perceives a stench in [x,y]
- Knowledge base
  - R_1: There is no pit in [1,1]:
  - A square is breezy iff there is a pit in adjacent squares
    - R_2: $B_{[1,1]}$
    - R_3: $B_{[2,1]}$
  - Percepts: no breeze in [1,1], breeze in [2,1]
    - R_4: and R_5:

Wumpus world in propositional logic



- Symbols
  - $P_{[x,y]}$ is true iff there is a pit in [x,y]
  - $W_{[x,y]}$ is true iff there is a wumpus in [x,y], dead or alive
  - $B_{[x,y]}$ is true iff the agent perceives a breeze in [x,y]
  - $S_{[x,y]}$ is true iff the agent perceives a stench in [x,y]
- Knowledge base
  - R_1: There is no pit in [1,1]: $\sim P_{[1,1]}$
  - A square is breezy iff there is a pit in adjacent squares
    - R_2: $B_{[1,1]} \Leftrightarrow (P_{[1,2]} \vee P_{[2,1]})$
    - R_3: $B_{[2,1]} \Leftrightarrow (P_{[1,1]} \vee P_{[2,2]} \vee P_{[3,1]})$
  - Percepts: no breeze in [1,1], breeze in [2,1]
    - R_4: $\sim B_{[1,1]}$ and R_5: $B_{[2,1]}$

Simple inference by enumeration

$KB = \{R_1, R_2, R_3, R_4, R_5\}$

- Query: “Is there a pit in [1,2] ?”
  - KB ⊨ $P_{[1,2]}$?
- Simplest inference: enumeration
  - Enumerate all models
  - Check if α is true in every model in which KB is true
- Propositional logic for wumpus world
  - 7 symbols ($B_{[1,1]}$, $B_{[2,1]}$, $P_{[1,1]}$, $P_{[1,2]}$, $P_{[2,1]}$, $P_{[2,2]}$, $P_{[3,1]}$) : 2^7 possible models
  - Compute the full truth table

Truth table for wumpus world

$KB = \{R_1, R_2, R_3, R_4, R_5\}$ is true in only 3 models.

$\neg P_{1,2}$ is true in all 3.

⇨ $KB \models \neg P_{1,2}$

$P_{2,2}$ is true in only 2 of 3. ⇨ No inference of $KB \models P_{2,2}$

128 rows

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	*false*	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*false*	*false*
false	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*false*	*true*	*false*	*false*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	*true*	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*false*	*true*	*true*	*false*
false	*true*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*false*	*true*	*false*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*true*	*false*	*false*	*true*	*false*	*false*	*true*	*true*	*false*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	*true*	*true*	*true*	*true*	*true*	*true*	*false*	*true*	*true*	*false*	*true*	*false*

Bad News

Suppose KB and α have n symbols. $\Longrightarrow$ 2^n models!

The propositional entailment problem of showing $KB \models \alpha$ by truth table enumeration requires

$\Theta(2^n n)$ time

$O(n)$ space (not bad)

The problem is co NP-complete (likely not easier than NP-complete).

Inference by enumeration

- Enumeration of all models is sound and complete.
- For *n* symbols, time complexity is $O(2^n)$...
- We need a smarter way to do inference!
- In particular, we are going to infer new logical sentences from the data-base and see if they match a query.

5/7. Theorem proving

- Inference by theorem proving
  - More efficient approach
  - Relies on logical equivalence: (α ≡ β)
    - Def 1: truth values for α and β are the same for a given model
    - Def 2: M(α) = M(β)
    - Example: α ∧ β ≡ β ∧ α

Theorem proving: Apply rules of inference directly to the sentences in KB to construct a proof of a sentence without consulting models.

Logical equivalences

- Two sentences are logically equivalent iff they are true in same models: α ≡ ß iff α ⊨ β and β ⊨ α

$$
\begin{aligned}
(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\
(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\
((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\
((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\
\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\
(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\
\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{De Morgan} \\
\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{De Morgan} \\
(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\
(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge
\end{aligned}
$$

You need to know these !

Validity and satisfiability

- Valid: the sentence is always true in all models
  - Example: $\alpha \vee \sim\alpha$
  - Any valid sentence is logically equivalent to True
- Satisfiable: the sentence is true in at least one model
  - Example: α
- Unsatisfiable: the sentence cannot be true in any model
  - Example: $\alpha \wedge \sim\alpha$
  - Any unsatisfiable sentence is logically equivalent to False
- Relationship: valid, satisfiable, unsatisfiable
  - α is valid iff $\sim\alpha$ is unsatifiable
  - α is satisfiable iff $\sim\alpha$ is not valid

- Entailment, validity, satisfiability, unsatisfiability
- $\alpha \models \beta$ iff
  - $(\alpha \Rightarrow \beta)$ is valid (=true in all models)
  - $\sim(\alpha \Rightarrow \beta)$ is unsatisfiable (=false in all models)
  - $\sim(\sim\alpha \vee \beta)$ is unsatisfiable (=false in all models)
  - $(\alpha \wedge \sim\beta)$ is unsatisfiable (=false in all models)

Validity is connected to inference via the Deduction Theorem:
$KB \models \alpha$ if and only if $(KB \Rightarrow \alpha)$ is valid

Satisfiability is connected to inference via the following:
$KB \models \alpha$ if and only if $(KB \wedge \neg\alpha)$ is unsatisfiable
(there is no model for which KB=true and α is false)

- Let U be the universe = the set of all models
- Lemmas you need for the proofs
  - $M(\sim\alpha) = U \setminus M(\alpha)$
  - $M(\alpha) \subseteq M(\beta)$ iff $M(\sim\beta) \subseteq M(\sim\alpha)$
  - $M(\alpha \wedge \beta) = M(\alpha) \cap M(\beta)$
  - $M(\alpha \vee \beta) = M(\alpha) \cup M(\beta)$
  - Let A,B,C be some sets. If $A \subseteq B$ then

    $A \cap C \subseteq B \cap C$, and also $A \cup C \subseteq B \cup C$

Inference rules

- There are several inference rules from the logical equivalences
- Generally written in this form:

  antecedent (or premise)

  consequent (or conclusion)

- If the KB contains the antecedent, we can add the consequent to KB

Inference rules

- Modus Ponens (MP)

  $\alpha \Rightarrow \beta, \quad \alpha$

  β

- Modus Tolens (MT)

  $\alpha \Rightarrow \beta, \quad \neg\beta$

  $\neg\alpha$

- Unit Resolution (UR)

  $\alpha \vee \beta, \quad \neg\beta$

  α

- AND introduction (AI)

  $\alpha, \quad \beta$

  $\alpha \wedge \beta$

- OR introduction (OI)

  α

  $\alpha \vee \beta$

- AND elimination (AE)

  $\alpha \wedge \beta$

  α, β

Note 1: α and β can be complex sentence
Note 2: AI joins two sentences into one, AE breaks a sentence into two.

Inference rules

Modus Ponens

$$\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$$

α $\Rightarrow$ β

If **today is Tuesday**, then **John will go to campus**.

Today is Tuesday.

Therefore, **John will go to campus**.

$(\alpha \wedge \beta)$	$\equiv$	$(\beta \wedge \alpha)$	commutativity of $\wedge$
$(\alpha \vee \beta)$	$\equiv$	$(\beta \vee \alpha)$	commutativity of $\vee$
$((\alpha \wedge \beta) \wedge \gamma)$	$\equiv$	$(\alpha \wedge (\beta \wedge \gamma))$	associativity of $\wedge$
$((\alpha \vee \beta) \vee \gamma)$	$\equiv$	$(\alpha \vee (\beta \vee \gamma))$	associativity of $\vee$
$\neg(\neg\alpha)$	$\equiv$	α	double-negation elimination
$(\alpha \Rightarrow \beta)$	$\equiv$	$(\neg\beta \Rightarrow \neg\alpha)$	contraposition
$(\alpha \Rightarrow \beta)$	$\equiv$	$(\neg\alpha \vee \beta)$	implication elimination
$(\alpha \Leftrightarrow \beta)$	$\equiv$	$((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$	biconditional elimination
$\neg(\alpha \wedge \beta)$	$\equiv$	$(\neg\alpha \vee \neg\beta)$	De Morgan
$\neg(\alpha \vee \beta)$	$\equiv$	$(\neg\alpha \wedge \neg\beta)$	De Morgan
$(\alpha \wedge (\beta \vee \gamma))$	$\equiv$	$((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$	distributivity of $\wedge$ over $\vee$
$(\alpha \vee (\beta \wedge \gamma))$	$\equiv$	$((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$	distributivity of $\vee$ over $\wedge$

$$\frac{\neg(\alpha \vee \beta)}{\neg\alpha \wedge \neg\beta}$$ De Morgan

$$\frac{\neg\alpha \wedge \neg\beta}{\neg\beta}$$ and-elimination

Inference rules: practice

- Modus Ponens:

  $\alpha \Rightarrow \beta,\ \alpha$

  β

- Modus Tolens:

  $\alpha \Rightarrow \beta,\ \sim\beta$

  $\sim\alpha$

- Unit Resolution:

  $\alpha \vee \beta,\ \sim\beta$

  α

- AND introduction

  $\alpha,\ \beta$

  $\alpha \wedge \beta$

- OR introduction:

  α

  $\alpha \vee \beta$

- AND elimination:

  $\alpha \wedge \beta$

  α, β

- KB:
  1. $P \wedge Q \Rightarrow R$
  2. $Q \vee S$
  3. $\sim S$
  4. $P \wedge N$

- $KB \models R$?

Wumpus world in propositional logic

- Knowledge base
  - R_1: $\sim P_{[1,1]}$
  - R_2: $B_{[1,1]} \Leftrightarrow (P_{[1,2]} \vee P_{[2,1]})$
  - R_3: $B_{[2,1]} \Leftrightarrow (P_{[1,1]} \vee P_{[2,2]} \vee P_{[3,1]})$
  - R_4: $\sim B_{[1,1]}$
  - R_5: $B_{[2,1]}$
- $KB \models \sim P_{[1,2]}$?

Wumpus world

- Goal: use R_1 … R_5 to prove $\sim P_{[1,2]}$
- Biconditional in $B_{[1,1]} \Leftrightarrow (P_{[1,2]} \vee P_{[2,1]})$

  R_6: $(B_{[1,1]} \Rightarrow (P_{[1,2]} \vee P_{[2,1]})) \wedge ((P_{[1,2]} \vee P_{[2,1]}) \Rightarrow B_{[1,1]})$
- And-Elimination to R_6

  R_7: $((P_{[1,2]} \vee P_{[2,1]}) \Rightarrow B_{[1,1]})$
- Contrapositive of R_7

  R_8: $\sim B_{[1,1]} \Rightarrow \sim(P_{[1,2]} \vee P_{[2,1]})$
- Modus Ponens with R_8 and percept R_4 $(\sim B_{[1,1]})$

  R_9: $\sim(P_{[1,2]} \vee P_{[2,1]})$
- De Morgan's rule

  R_{10}: $\sim P_{[1,2]} \wedge \sim P_{[2,1]}$
- And-Elimination: $\sim P_{[1,2]}$

6/7. Resolution

- Recall *unit resolution:*

  $\underline{A \vee B, \quad \neg B}$

  A

Resolution: inference rule for CNF: sound and complete!

- Can be generalized into:

  $\underline{A \vee B \vee C \ldots \vee F \vee G, \qquad \neg G \vee H \vee I \vee \ldots \vee Z}$

  $A \vee B \vee C \ldots \vee F \vee H \vee I \vee \ldots \vee Z$

- Example: wumpus world

  $\underline{P_{[1,1]} \vee P_{[3,1]}, \qquad \neg P_{[1,1]} \vee \neg P_{[2,2]}}$

  $P_{[3,1]} \vee \neg P_{[2,2]}$

- Factoring: remove multiple copies of literals
  - $(A \vee B), (A \vee \neg B) \equiv (A \vee A) \equiv A$

Conjunctive normal form

- Resolution requires sentences in CNF
  - Write a sentence as a conjunction of clauses
  - Clause is a disjunction of literals
  - Every sentence in PL can be written in CNF
- Example: write $B_{[1,1]} \Leftrightarrow (P_{[1,2]} \vee P_{[2,1]})$ in CNF
  - Eliminate biconditional:

    $(B_{[1,1]} \Rightarrow (P_{[1,2]} \vee P_{[2,1]})) \wedge ((P_{[1,2]} \vee P_{[2,1]}) \Rightarrow B_{[1,1]})$
  - Replace $A \Rightarrow B$ with $\sim A \vee B$:

    $(\sim B_{[1,1]} \vee P_{[1,2]} \vee P_{[2,1]}) \wedge (\sim(P_{[1,2]} \vee P_{[2,1]}) \vee B_{[1,1]})$
  - De Morgan's rule:

    $(\sim B_{[1,1]} \vee P_{[1,2]} \vee P_{[2,1]}) \wedge ((\sim P_{[1,2]} \wedge \sim P_{[2,1]}) \vee B_{[1,1]})$
  - Distribute $\vee$ over $\wedge$

    $(\sim B_{[1,1]} \vee P_{[1,2]} \vee P_{[2,1]}) \wedge (\sim P_{[1,2]} \vee B_{[1,1]}) \wedge (\sim P_{[2,1]} \vee B_{[1,1]})$

Proof by resolution

- Resolution rule alone can be used for inference (without MP, MP, AI, OI, or AE)
  - *Sound* and *complete* inference
- Resolution algorithm
  - To show KB ⊨ α, show (KB ∧ ~α) is unsatisfiable
  - Proof by contradiction
  - First convert (KB ∧ ~α) into CNF
  - Apply resolution to each pair: add the new clause to the set
  - Process stops when
    - When no new clause is created: KB does not entail α
    - Or, if two clauses resolve to the empty clause: KB entails α
    - Why? Empty clause is False: P, ~P, therefore (KB ∧ ~α) is unsatisfiable

Proof by resolution

We like to prove: $KB \models \alpha$

equivalent to : $KB \wedge \neg\alpha$ *unsatifiable*

We first rewrite $KB \wedge \neg\alpha$ **into conjunctive normal form (CNF).**

A "conjunction of disjunctions"

literals

$$(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$$

Clause **Clause**

- **Any KB can be converted into CNF.**
- **In fact, any KB can be converted into CNF-3 using clauses with at most 3 literals.**

A simple resolution algorithm for PL

function PL-RESOLUTION(KB, α) **returns** *true* or *false*

inputs: KB, the knowledge base, a sentence in propositional logic
α, the query, a sentence in propositional logic

$clauses \leftarrow$ the set of clauses in the CNF representation of $KB \wedge \neg\alpha$
$new \leftarrow \{\}$
loop do
  for each pair of clauses C_i, C_j **in** $clauses$ **do**
    $resolvents \leftarrow$ PL-RESOLVE(C_i, C_j)
    if $resolvents$ contains the empty clause **then return** *true*
    $new \leftarrow new \cup resolvents$
  if $new \subseteq clauses$ **then return** *false*
  $clauses \leftarrow clauses \cup new$

Resolution algorithm for PL

- **The resolution algorithm tries to prove:**

$KB \models \alpha$ *equivalent to*

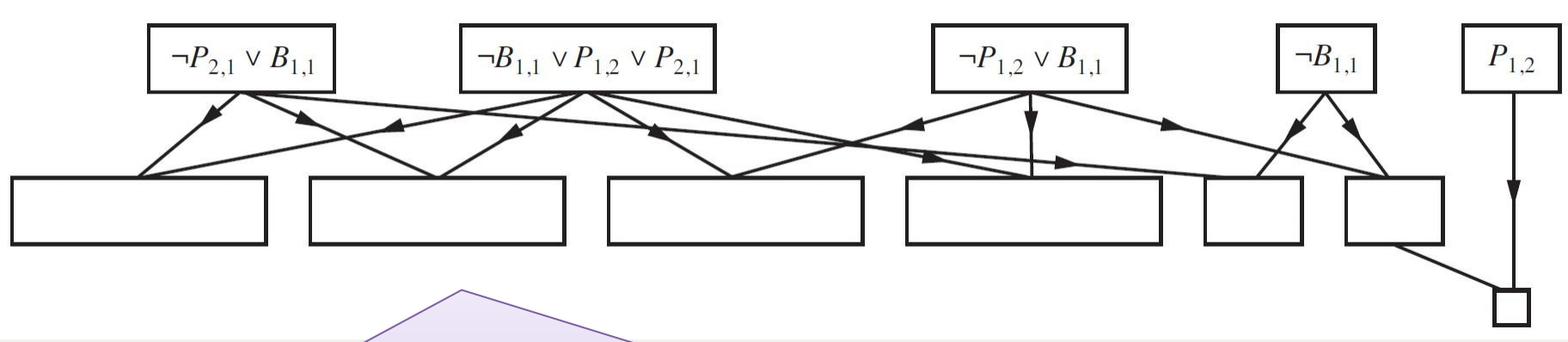
$KB \wedge \neg\alpha$ *unsatisfiable*

- **Generate all new sentences from KB and the query.**
- **One of two things can happen:**

1. **We find** $P \wedge \neg P$ **which is unsatisfiable. I.e. we can entail the query.**

2. **We find no contradiction: there is a model that satisfies the sentence** $KB \wedge \neg\alpha$ **(non-trivial) and hence we cannot entail the query.**

- PL-resolution in the wumpus world



We can apply the resolution procedure to a very simple inference in the wumpus world. When the agent is in [1,1], there is no breeze, so there can be no pits in neighboring squares. The relevant knowledge base is

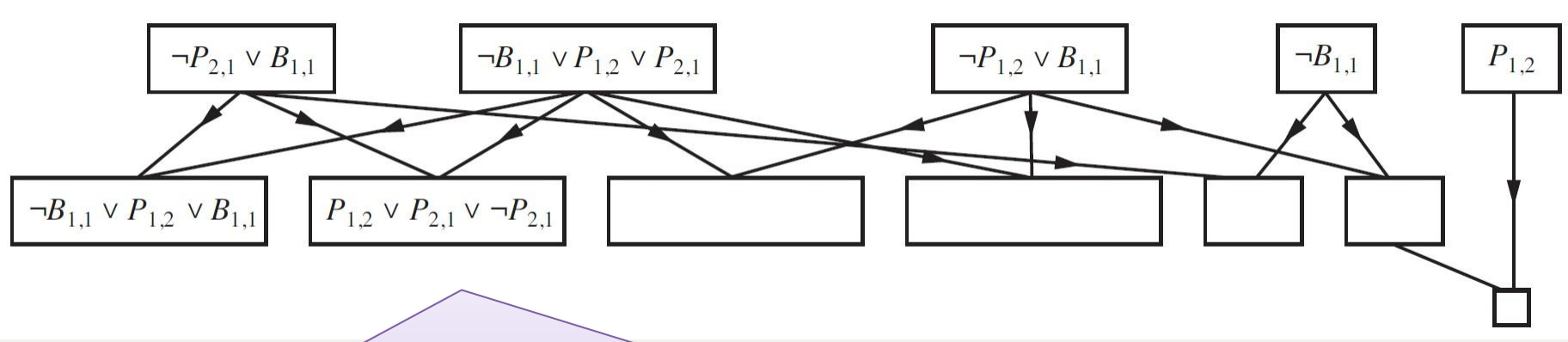
$$KB = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1}$$

and we wish to prove α *which is, say,*

$$\alpha = \neg P_{1,2}$$

When we convert (KB ^ ¬a) into CNF, we obtain the clauses shown at the top of the above figure . The second row of the figure shows clauses obtained by resolving pairs in the first row.

- PL-resolution in the wumpus world



We can apply the resolution procedure to a very simple inference in the wumpus world. When the agent is in [1,1], there is no breeze, so there can be no pits in neighboring squares. The relevant knowledge base is

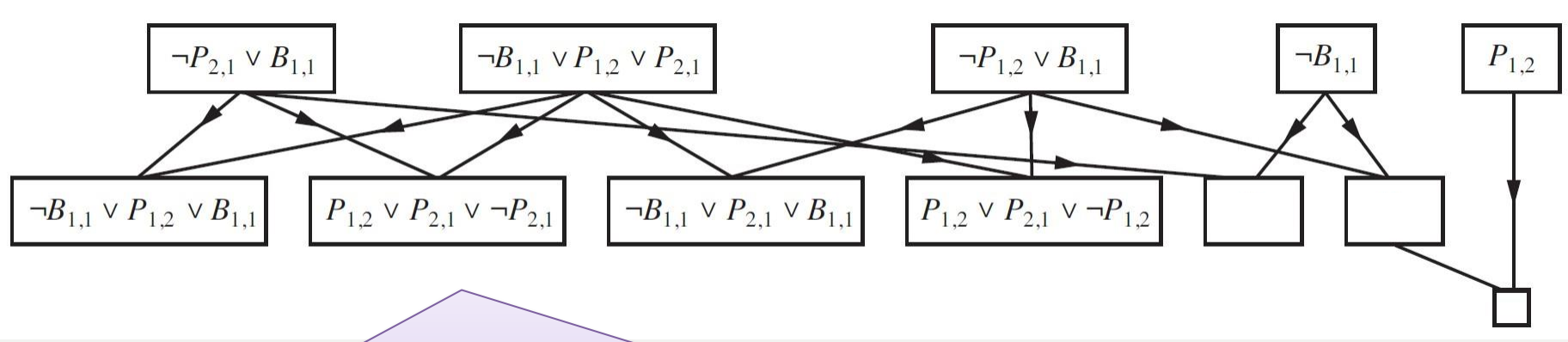
$$KB = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1}$$

and we wish to prove α *which is, say,*

$$\alpha = \neg P_{1,2}$$

When we convert (KB ^ ¬a) into CNF, we obtain the clauses shown at the top of the above figure . The second row of the figure shows clauses obtained by resolving pairs in the first row.

- PL-resolution in the wumpus world



We can apply the resolution procedure to a very simple inference in the wumpus world. When the agent is in [1,1], there is no breeze, so there can be no pits in neighboring squares. The relevant knowledge base is

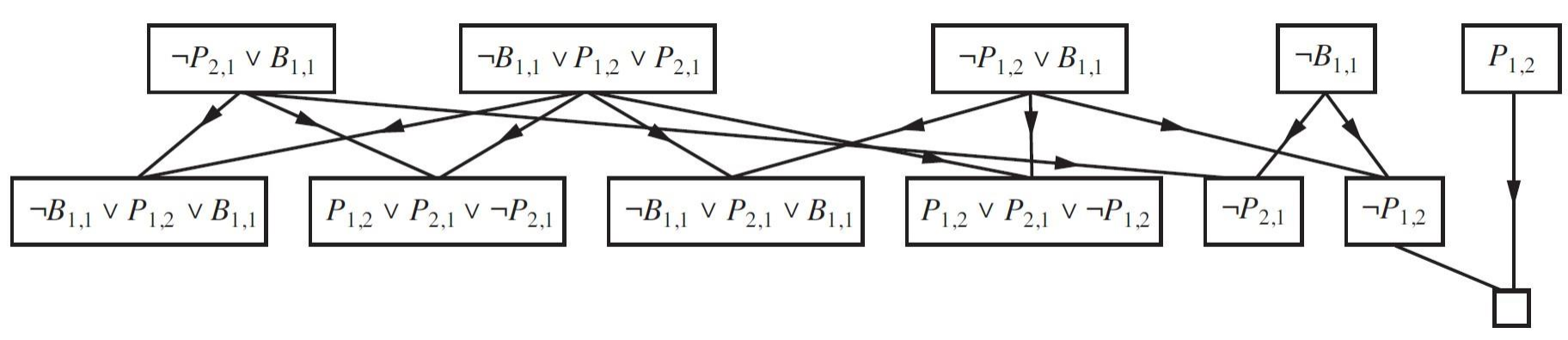
$$KB = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1}$$

and we wish to prove α *which is, say,*

$$\alpha = \neg P_{1,2}$$

When we convert (KB ^ ¬a) into CNF, we obtain the clauses shown at the top of the above figure . The second row of the figure shows clauses obtained by resolving pairs in the first row.

- PL-resolution in the wumpus world



- Resolve $P_{[1,2]}$, $\sim P_{[1,2]}$: { } = *False*
- Conclude: KB ⊨ $\sim P_{[1,2]}$

Resolution practice

- KB:
  1. P v Q v Y
  2. Q v J
  3. ~J v C
  4. ~Q
  5. ~Y v ~C v P
- KB ⊨ P ?
  - Check CNF
  - Add ~P to KB

Resolution practice

- KB:
  1. P v Q v Y
  2. Q v J
  3. ~J v C
  4. ~Q
  5. ~Y v ~C v P
  6. ~P

6. ~P (given)
7. P v Y (1 and 4)
8. ~C v P (7 and 5)
9. ~C (6 and 8)
10. ~J (9 and 3)
11. Q (2 and 10)

Contradiction! 11 and 4!

Q ∧ ~Q

Forward/backward chaining (optional)

- Two other inference algorithms (optional)
  - Forward chaining: incrementally insert sentences into the set, and derive all possible consequences
  - Backward chaining: work backward from the conclusion by checking if antecedents are true
  - More efficient than resolution, with restrictions
    - All sentences has to be written in definite clauses
    - Defs:
      - Definite clause: disjunction of literals of which exactly one is positive
      - Horn clause: disjunction of literals of which at most one is positive

Forward/backward chaining (optional)

- AND-OR graph for forward/backward chaining
- KB in Horn clauses. Does KB ⊨ Q?

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

$B \wedge L \Rightarrow M$

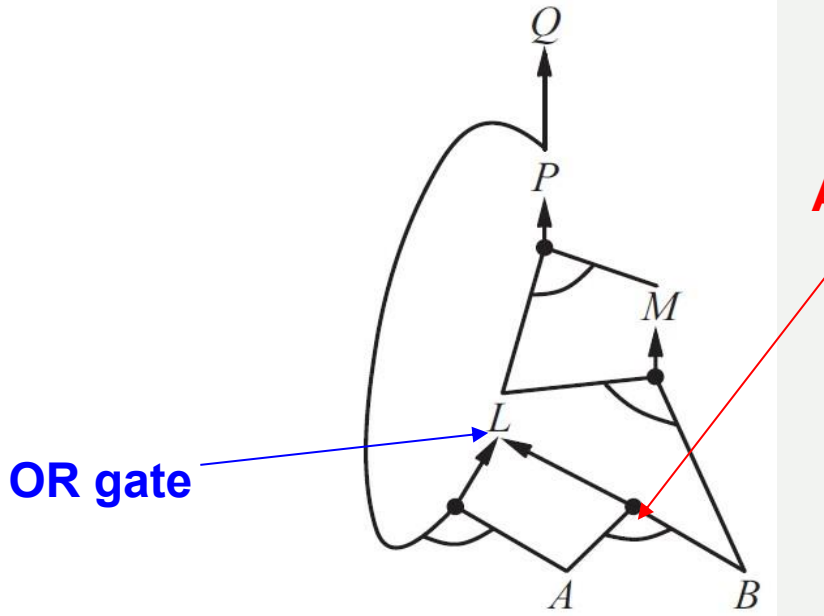
$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L$

A

B

(a)

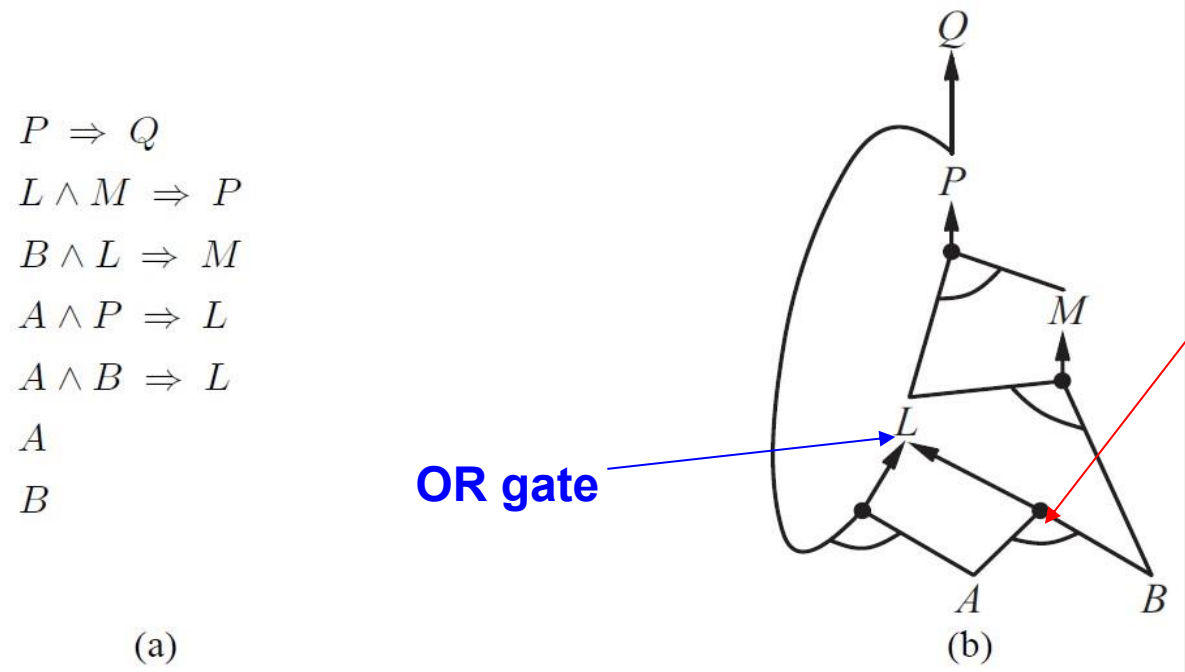


(b)

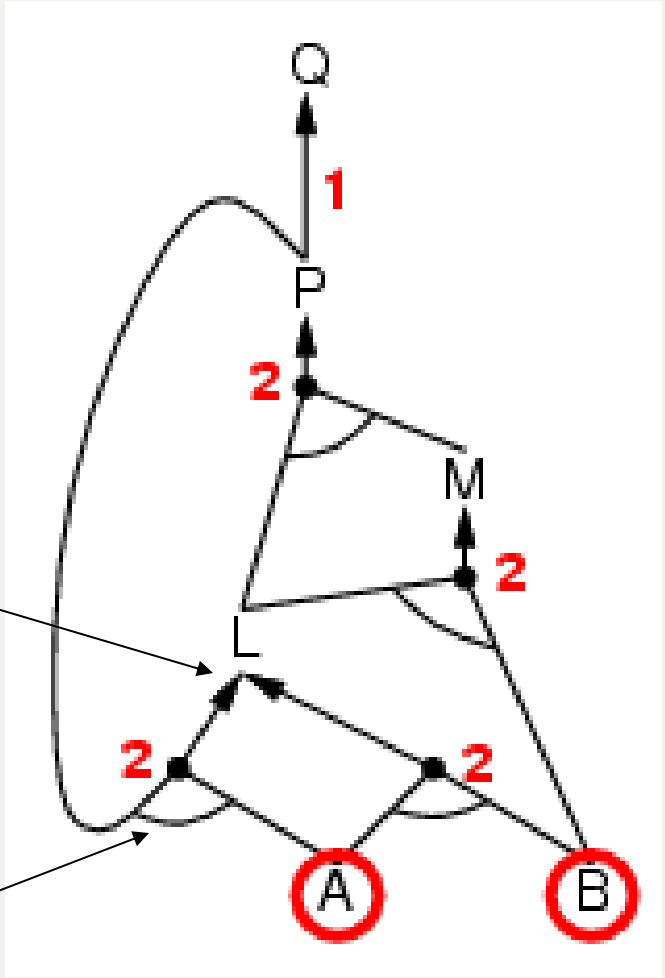
Forward chaining (optional)

- AND-OR graph for forward
  - Idea: fire any rule whose premises are satisfied in the *KB*,
  - add its conclusion to the *KB*, until query is found

** (a) shows a simple knowledge base of Horn clauses with A and B as known fact. (b) shows the same knowledge base drawn as an AND-OR graph. Multiple links joined by an arc indicate a conjunction-every link must be proved-while multiple links without an arc indicate a disjunction-any link can be proved.*
** The known leaves (here, A & B) are set, and inference propagates up the graph as far as possible*



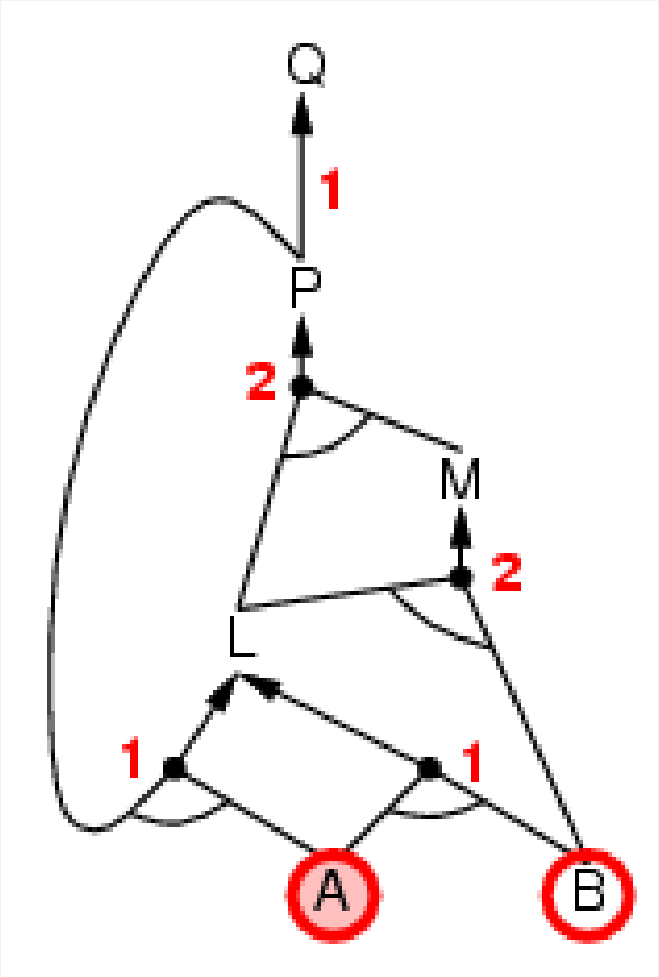
Forward chaining example



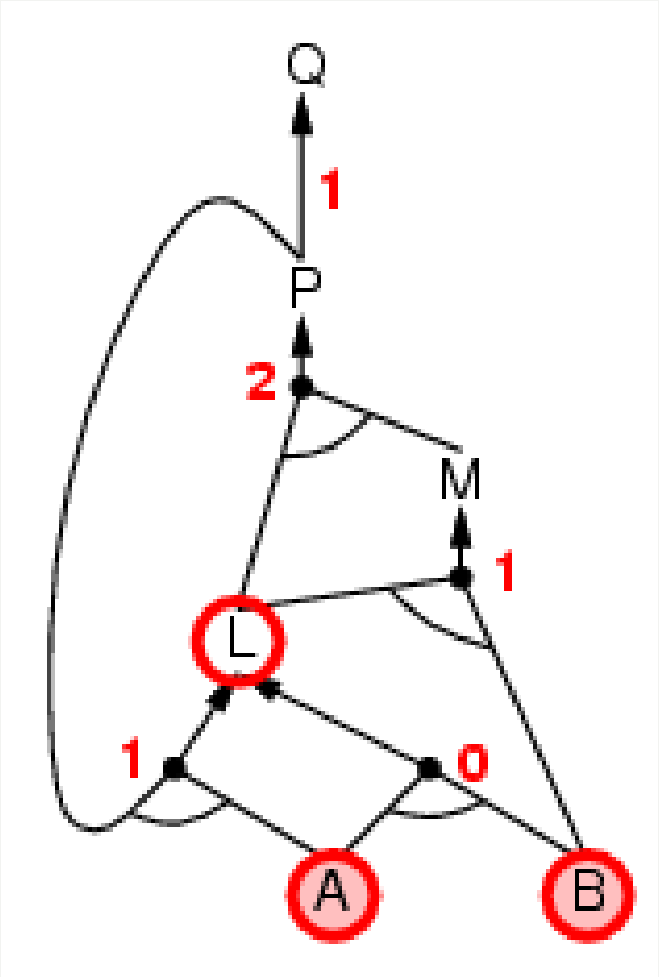
“OR” Gate

“AND” gate

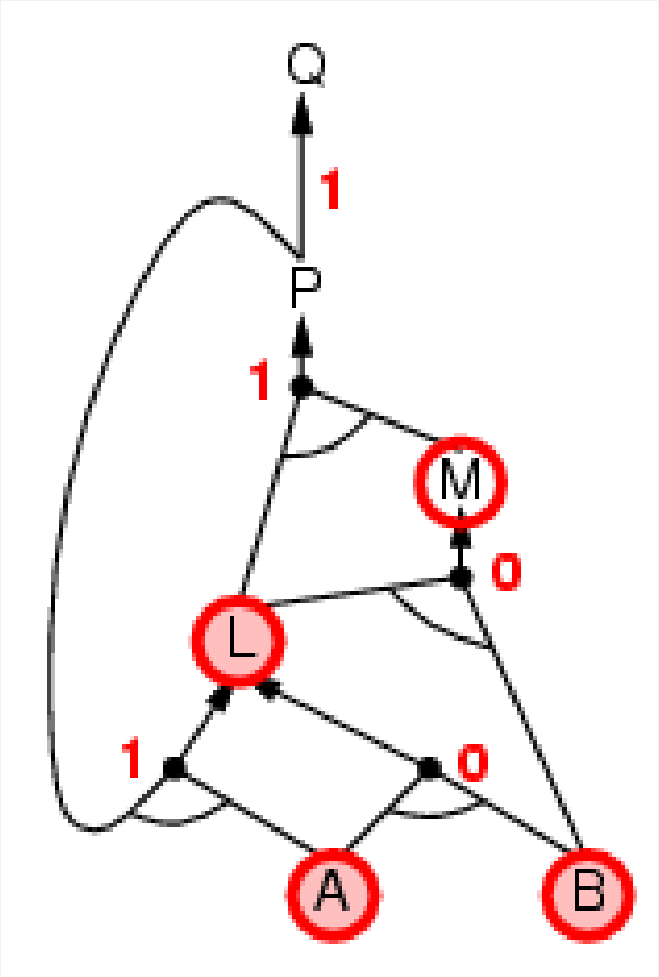
Forward chaining example



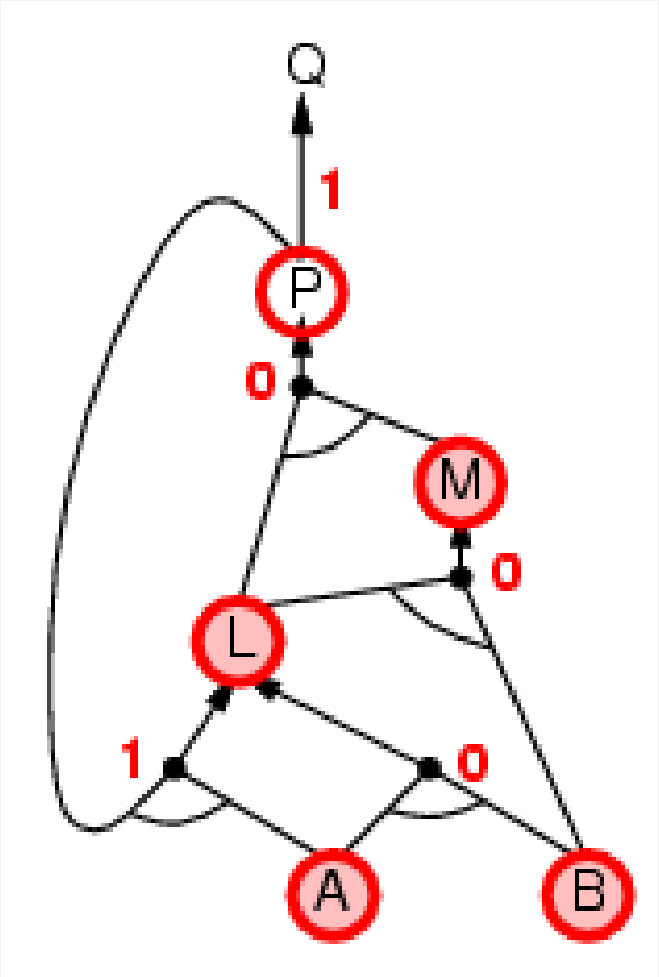
Forward chaining example



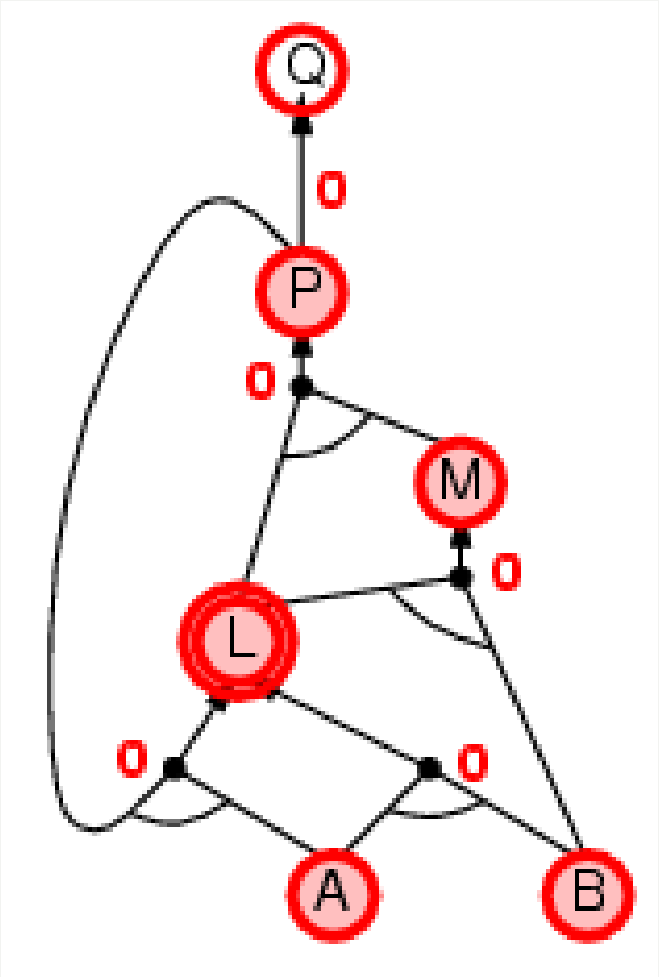
Forward chaining example



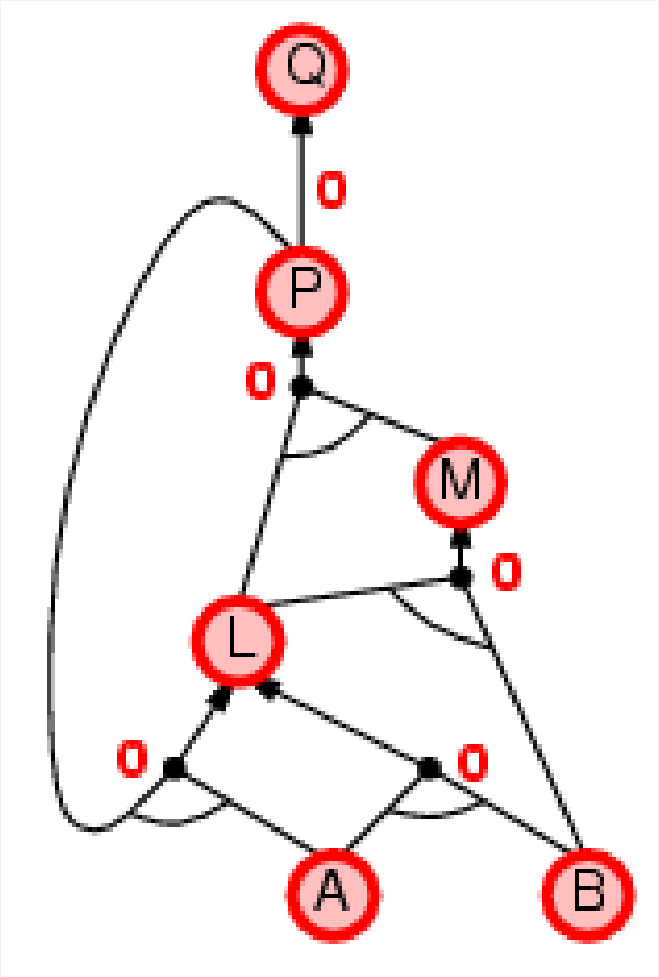
Forward chaining example



Forward chaining example



Forward chaining example



Backward chaining (optional)

Idea: work backwards from the query *q*

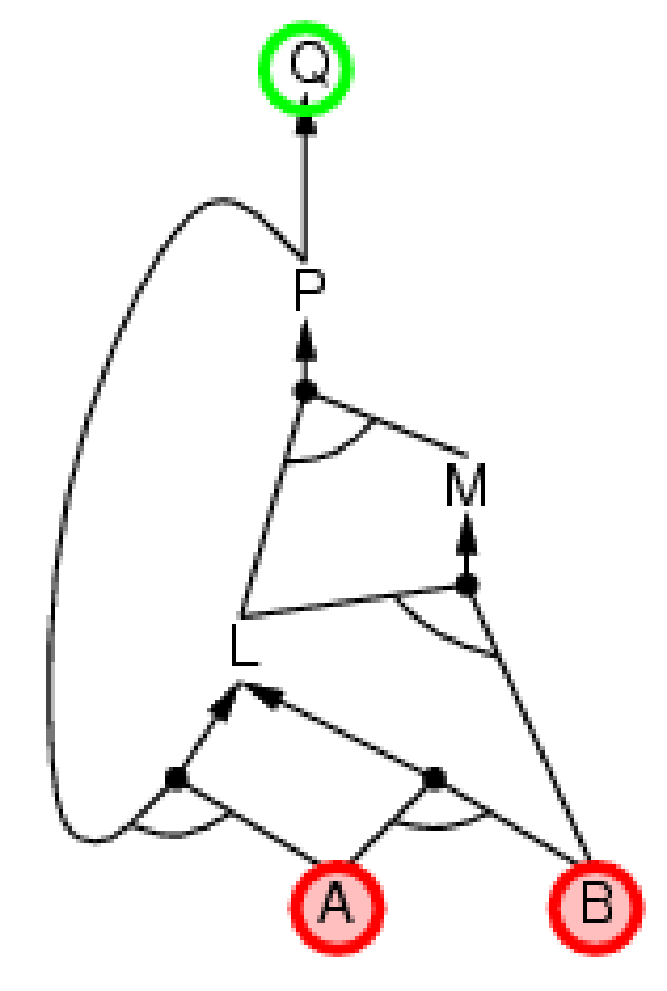
- check if *q* is known already, or
- prove by BC all premises of some rule concluding *q*
- Hence BC maintains a stack of sub-goals that need to be proved to get to q.

Avoid loops: check if new sub-goal is already on the goal stack

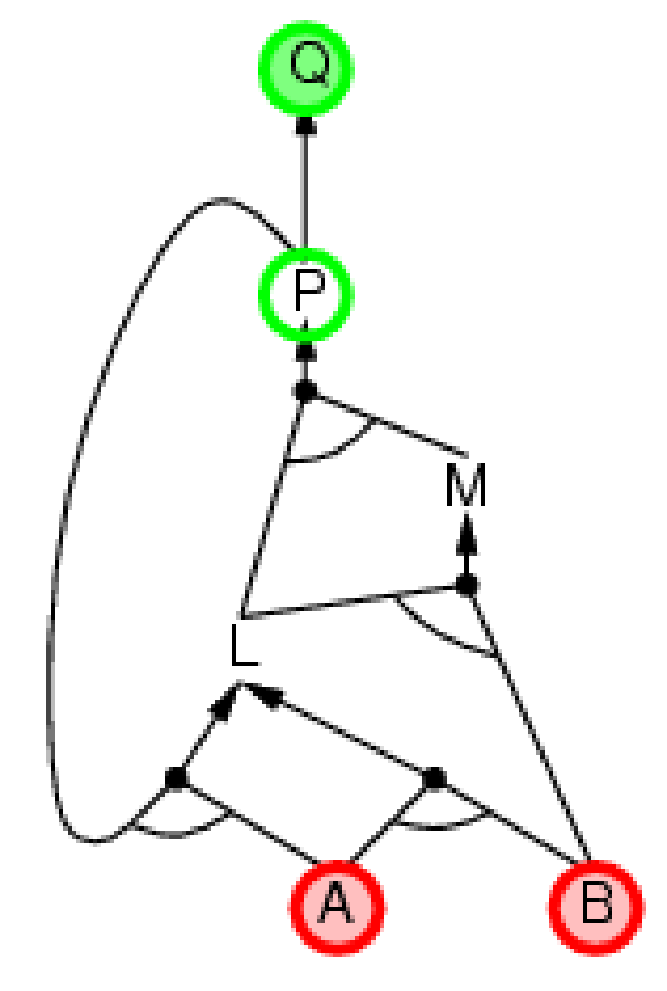
Avoid repeated work: check if new sub-goal

1. has already been proved true, or
2. has already failed

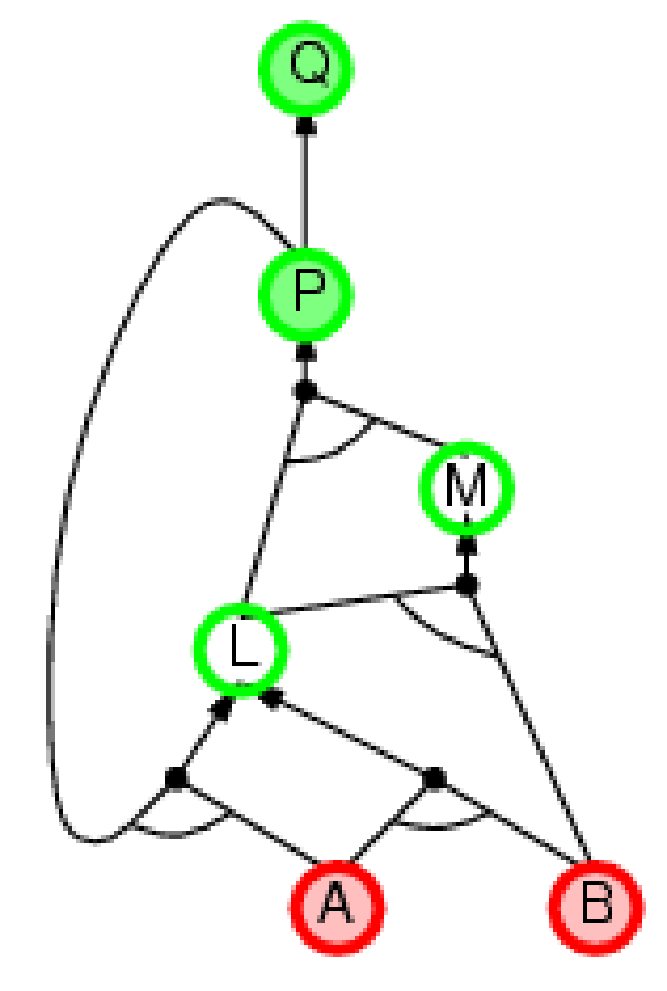
Backward chaining example



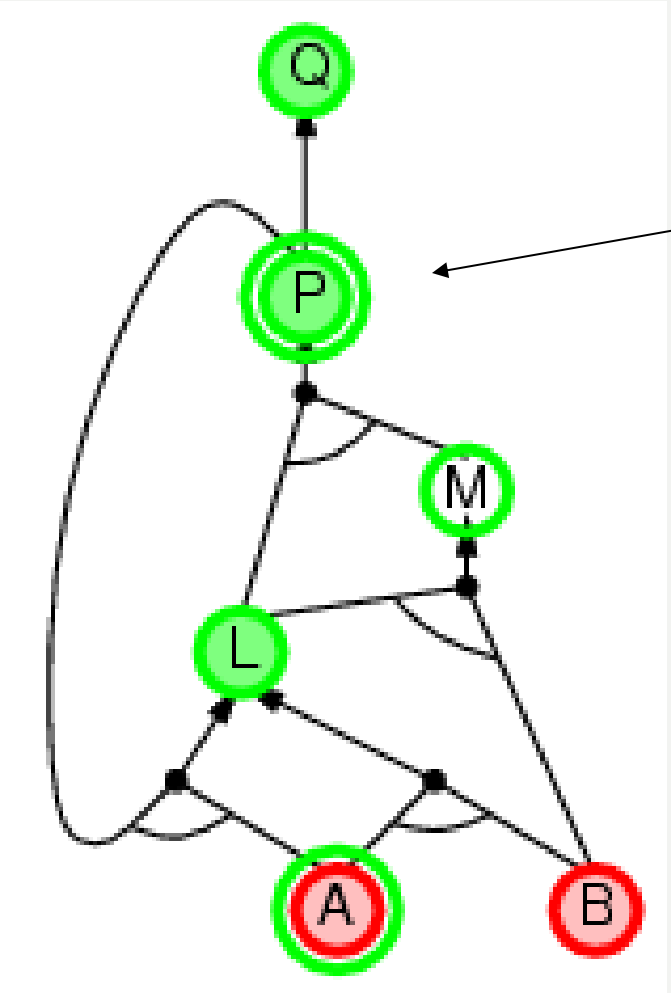
Backward chaining example



Backward chaining example

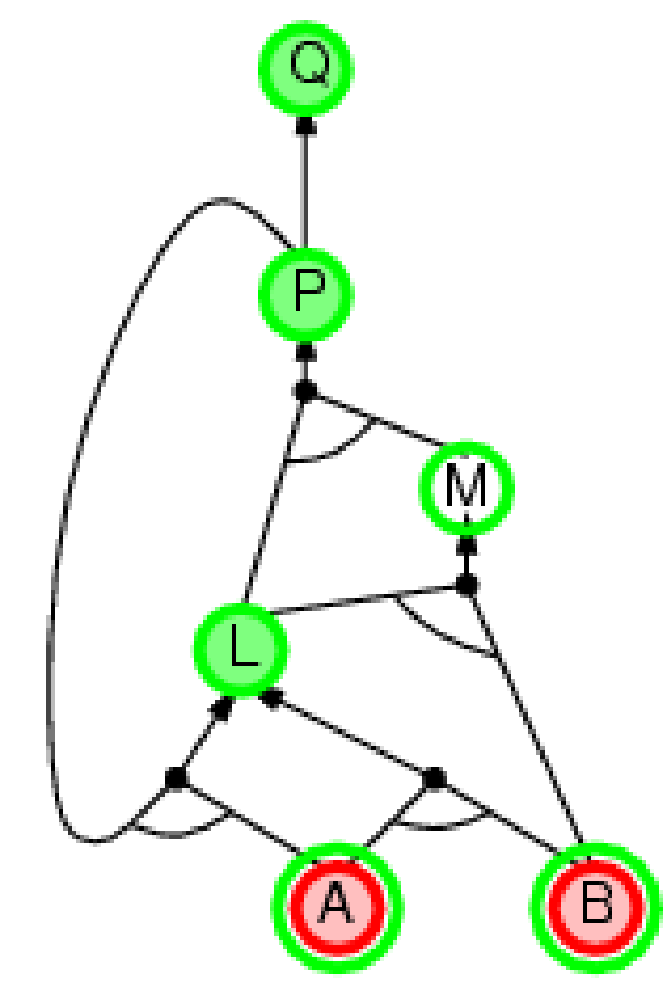


Backward chaining example

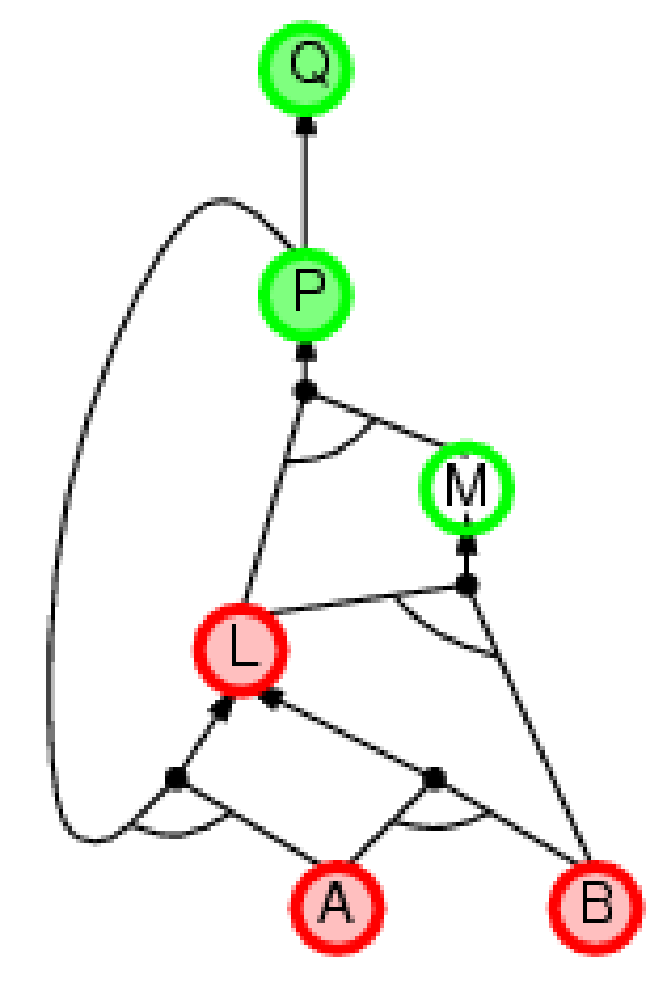


we need P to prove
L and L to prove P.

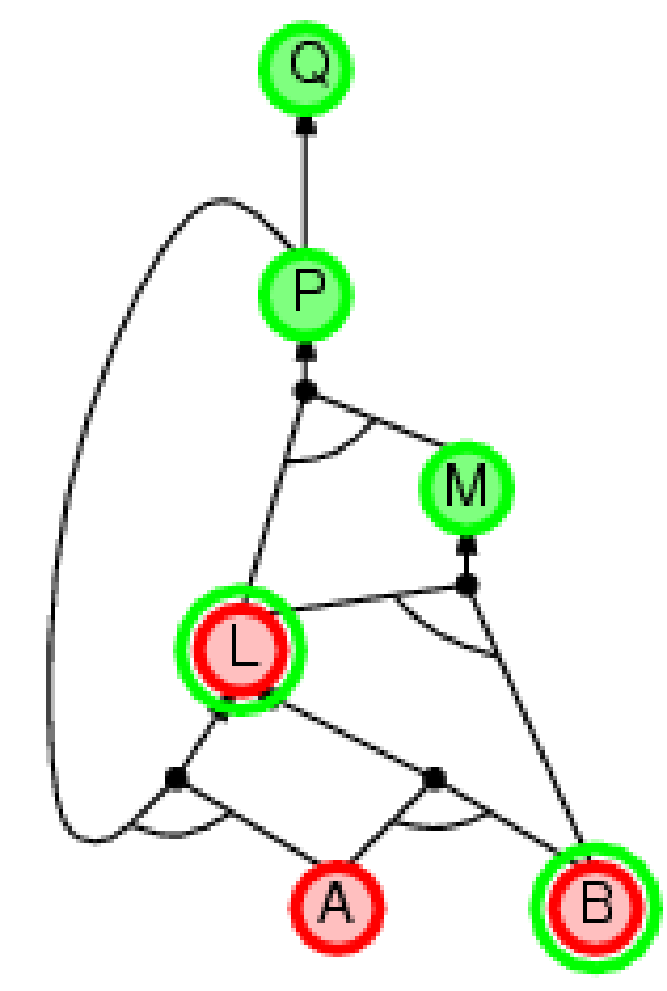
Backward chaining example



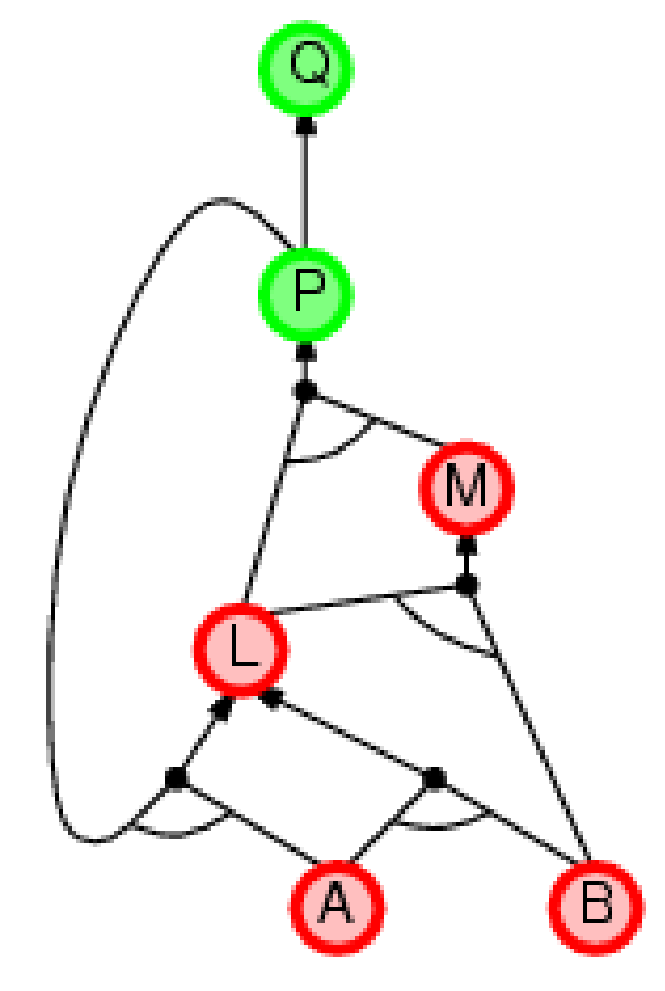
Backward chaining example



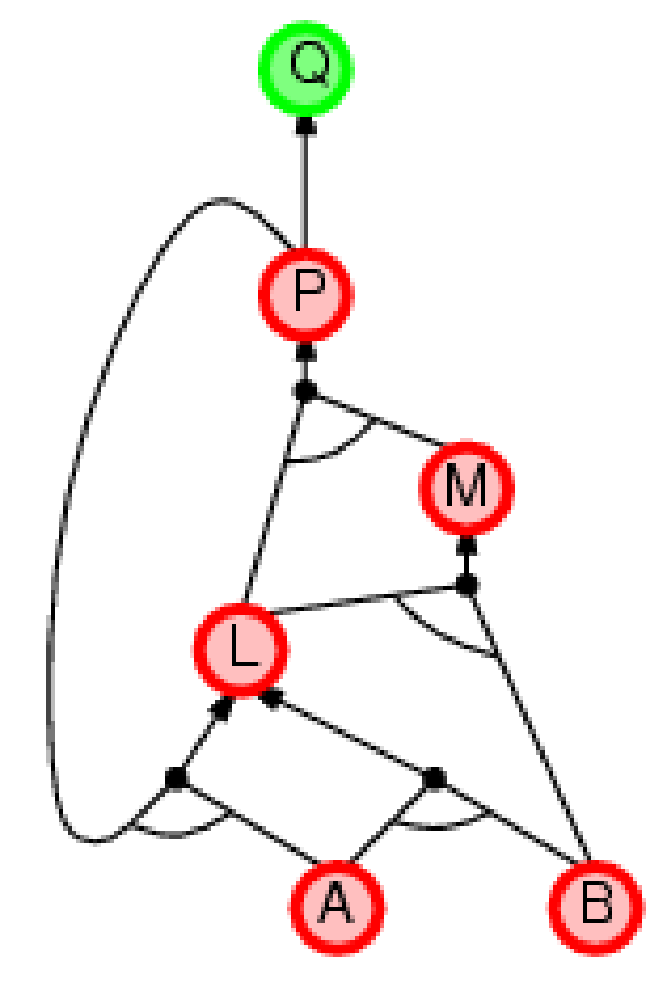
Backward chaining example



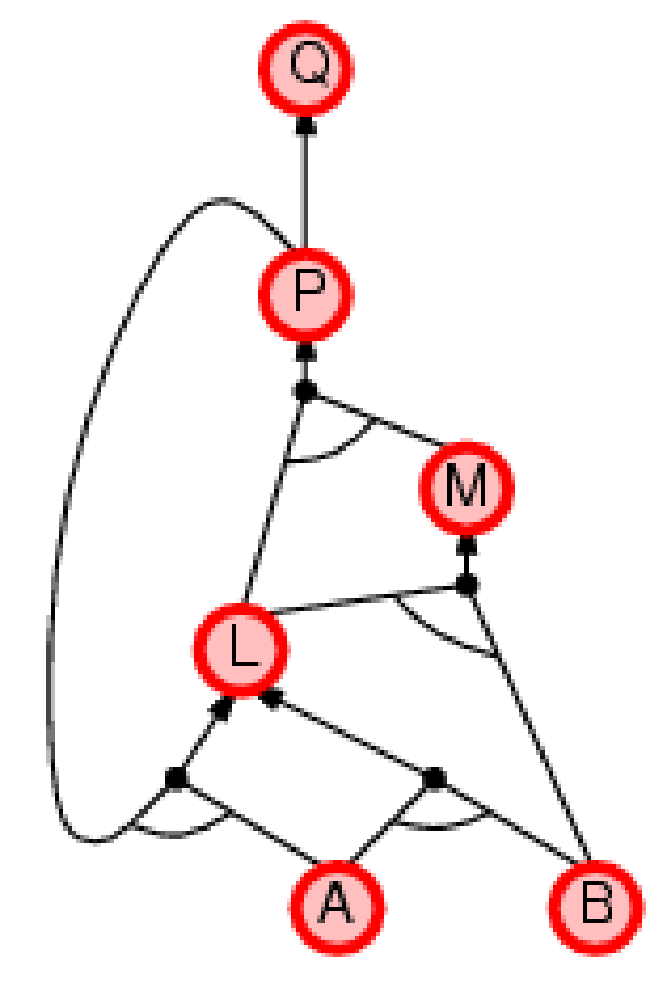
Backward chaining example



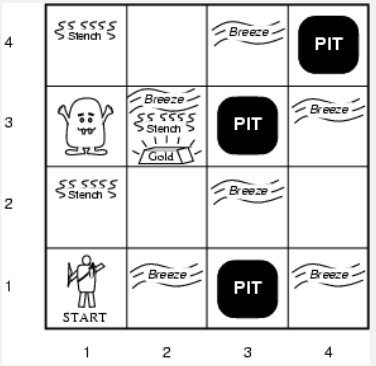
Backward chaining example



Backward chaining example



7/7. PL agents for wumpus world



- Axioms of the environment
  - No pit or wumpus at [1,1]: $\sim P_{[1,1]}$, $\sim W_{[1,1]}$,
  - $B_{[1,1]} \Leftrightarrow (P_{[1,2]} \vee P_{[2,1]})$
  - $S_{[1,1]} \Leftrightarrow (W_{[1,2]} \vee W_{[2,1]})$
  - …. repeat for all squares
  - Exactly one wumpus
    - At least one wumpus: $W_{[1,1]} \vee W_{[1,2]} \ldots W_{[4,3]} \vee W_{[4,4]}$
    - At most one wumpus: for any pair, at least one of them has to be wumpus-free.

    $(\sim W_{[1,1]} \vee \sim W_{[1,2]}) \wedge \ \ldots \ \wedge \ (\sim W_{[4,3]} \vee \sim W_{[4,4]})$

PL agents for wumpus world

- Transition (=effect) axioms: what changes by actions
  - $L^t_{[x,y]}$: agent is in [x,y] at time t
  - $L^0_{[1,1]} \wedge \text{FacingEast}^0 \wedge \text{Forward}^0 \Rightarrow L^1_{[2,1]} \wedge \sim L^1_{[1,1]}$
  - … repeat for all squares, all time steps, all actions such as (Grab, Shoot, Climb, TurnLeft, TurnRight)
- Frame problem: we also have to specify what doesn't change by actions
  - What happens to HaveArrow^1 ?
  - $\text{Forward}^t \Rightarrow (\text{HaveArrow}^t \Leftrightarrow \text{HaveArrow}^{t+1})$
  - $\text{Forward}^t \Rightarrow (\text{WumpusAlive}^t \Leftrightarrow \text{WumpusAlive}^{t+1})$
  - … repeat for all fluents and atemporal states
  - Fluents: states that change over time, atemporal: that doesn't.

- Success-state axioms: for each fluent
  - $F^{t+1} \Leftrightarrow \text{ActionCausesF}^t \vee (F^t \wedge \sim\text{ActionCausesNotF}^t)$
  - Ex: $\text{HaveArrow}^{t+1} \Leftrightarrow (\text{HaveArrow}^t \wedge \sim\text{Shoot}^t)$
  - $L^{t+1}_{[1,1]} \Leftrightarrow (L^t_{[1,1]} \wedge (\sim\text{Forward}^t \vee \text{Bump}^{t+1}))$
    $\vee (L^t_{[1,2]} \wedge (\text{South}^t \vee \text{Forward}^t))$
    $\vee (L^t_{[2,1]} \wedge (\text{West}^t \vee \text{Forward}^t))$
  - … repeat for all locations, all time steps, and all other fluents
- Finally,
  - Agent can ASK its KB whether certain squares are safe to move, contains gold, etc
  - $\text{OK}^{t+1}_{[x,y]} \Leftrightarrow \sim P_{[x,y]} \wedge \sim(W_{[x,y]} \wedge \text{WumpusAlive}^t)$
  - Solving the wumpus problem with PL is extremely inconvenient !

The solution to the problem involves changing one's focus from writing axioms about *actions* to writing axioms about *fluents*. Thus, for each fluent F, we will have an axiom that defines the truth value of F^{t+1} in terms of fluents (including F itself) at time t and the actions that may have occurred at time t. Now, the truth value of F^{t+1} can be set in one of two ways: either the action at time t causes F to be true at $t+1$, or F was already true at time t and the action at time t does not cause it to be false. An axiom of this form is called a **successor-state axiom** and has this schema:

$$F^{t+1} \Leftrightarrow ActionCausesF^t \vee (F^t \wedge \neg ActionCausesNotF^t) .$$

One of the simplest successor-state axioms is the one for $HaveArrow$. Because there is no action for reloading, the $ActionCausesF^t$ part goes away and we are left with

$$HaveArrow^{t+1} \Leftrightarrow (HaveArrow^t \wedge \neg Shoot^t) .$$

For the agent's location, the successor-state axioms are more elaborate. For example, $L_{1,1}^{t+1}$ is true if either (a) the agent moved *Forward* from $[1,2]$ when facing south, or from $[2,1]$ when facing west; or (b) $L_{1,1}^t$ was already true and the action did not cause movement (either because the action was not $Forward$ or because the action bumped into a wall). Written out in propositional logic, this becomes

$$\begin{aligned} L_{1,1}^{t+1} \Leftrightarrow\ & (L_{1,1}^t \wedge (\neg Forward^t \vee Bump^{t+1})) \\ \vee\ & (L_{1,2}^t \wedge (South^t \wedge Forward^t)) \\ \vee\ & (L_{2,1}^t \wedge (West^t \wedge Forward^t)) . \end{aligned}$$

Announcement